Bioinformatics

2025/2026

Day 6

Exercise 1

Use conda to create an environment named bioinfo_lab6 that includes the latest version of samtools and the necessary software for all exercises. Create an environment.yml file with the versions used.

Exercise 2

Calculate the edit distance between sequence A = GATTACA and sequence B = GTCGACG using the dynamic programming matrix.

2.1 Initialise and fill the DP matrix manually (on paper or a spreadsheet) using a penalty of 1 for insertions, deletions, and substitutions.

2.2 Perform the traceback from the bottom-right cell to the top-left cell and write down the optimal alignment(s).

Exercise 3

Write a Python script **needleman_wunsch.py** that implements the Needleman-Wunsch algorithm for global sequence alignment using Dynamic Programming.

- **Algorithm:** O(n m) time and space complexity.
- **Input:** Accept two sequences as command-line arguments using argparse (e.g., --seq1 GATTACA --seq2 GTCGACG). Include optional arguments for the scoring system: --match (default 1), --mismatch (default -1), and --gap (default -1).
- **Output:** Print the final alignment score and the optimal alignment strings (representing gaps with a -) to standard output.
  Challenge: Have the script print the formatted DP scoring matrix and the traceback path before outputting the final alignment.

Exercise 4

You are comparing two short protein sequences:

- **Seq 1:** HEAGAWGHEE
- **Seq 2:** PAWHEAE

4.1 Use the EMBOSS needle tool (Needleman-Wunsch algorithm) to perform a global alignment. Use the BLOSUM62 matrix, a gap opening penalty of 10, and a gap extension penalty of 0.5. What is the alignment score?

4.2 Use the EMBOSS water tool (Smith-Waterman algorithm) to perform a local alignment using the exact same parameters. What is the optimal local alignment score?

4.3 Explain the biological significance of the difference between the global and local alignment results for these two sequences.

Exercise 5

Explain the fundamental heuristic trade-off used by high-throughput aligners like BLAST or Minimap2. By using a heuristic (like k-mer seeds), what do you computationally gain, and what do you lose compared to Smith-Waterman?

Exercise 6

You are analysing an RNA-Seq BAM file and extracting the following CIGAR string for a read mapped to the human reference genome: 30M1500N70M.

6.1 **Read Length:** Calculate the exact length of the raw sequencing read. Show your arithmetic.

6.2 **Reference Span:** Calculate the exact number of base pairs this read spans across the reference genome. Show your arithmetic.

6.3 **Biological Context:** What does the N operator represent biologically in this context, and how does it differ fundamentally from a D (deletion)?

6.4 **Algorithm Selection:** Why would a standard contiguous DNA aligner (like bwa mem) fail or incorrectly clip this specific read?

Exercise 7

7.1 Describe the primary algorithmic paradigm shift between a standard reference-based quantification pipeline (e.g., mapping with STAR followed by counting with featureCounts) and a pseudo-aligner (e.g., Kallisto or Salmon).

7.2 What is the main practical limitation of using a pseudo-aligner? (i.e., what specific data or positional information do you lose by bypassing base-level alignment?)

Exercise 8

8.1 Data Retrieval: Use a command-line tool (e.g., wget) to download the reference files from Ensembl/ENA.

- **Reference Genome:** Download the DNA FASTA file for the human genome (latest version).
- **Reference Transcriptome:** Download the corresponding cDNA FASTA file.
- **Reference Annotation:** Download the GTF annotation file that maps all known genes and exons.
- **Raw Reads:** Download paired-end reads from a public human RNA-seq experiment - SRR1039508.

If you are running this exercise on a computer with insufficient resources (e.g., memory), you may need to subsample the data:

- Subset the References: Instead of downloading the entire human genome, download the DNA FASTA (genome), cDNA FASTA (transcriptome), and GTF (annotation) files exclusively for Human Chromosome 22 (chr22).
- Subset the Reads: Instead of using the entire sequencing run, extract only the first 200,000 paired-end reads.

8.2 Indexing Strategy

8.2.1 Write the basic bash commands to build the index for **HISAT2** and the index for **Kallisto**.

8.2.2 Which FASTA file must you use for HISAT2, and which must you use for Kallisto?

8.2.3 It is highly recommended to extract splice sites from your **GTF file** to feed into the HISAT2 indexer. Why does Kallisto completely ignore the GTF file during its indexing phase?

8.3 Run the paired-end reads through both pipelines.

8.3.1 **HISAT2:** Map the reads to generate a .sam file, convert it to .bam, and conceptually explain how you would use the **GTF file** alongside featureCounts to get your final gene counts.

8.3.2 **Kallisto:** Quantify the reads directly against the transcriptome to generate your output .tsv file.

8.4 You suspect that this cell line features a completely **novel, unannotated alternative splicing event** (an exon-exon junction that is *not* in your GTF file). Which pipeline (HISAT2 or Kallisto) is absolutely required to discover this novel splice junction, and why does the other tool fail at this specific task?

8.5 Pipeline Concordance

It is standard practice to validate a newer, heuristic-based pipeline against an established reference pipeline. You now have quantification results from both `featureCounts` (based on exact genomic alignments) and `Kallisto` (based on heuristic pseudo-alignment).

8.5.1 Open both output files. You will notice Kallisto outputs *transcript-level* IDs (e.g., `ENST...`), whereas `featureCounts` outputs *gene-level* IDs (e.g., `ENSG...`). Which specific data manipulation step must you perform on the Kallisto data to accurately compare the two datasets?

8.5.2 Briefly explain the scripting steps (e.g., in Python or R) you would take to compare the estimated read counts between the two tools. What specific statistical metric(s) and visual plot would you use to evaluate their similarity?

8.5.3 Challenge

Write a script (in Python using pandas or R using dplyr/ggplot2) to actively perform the comparison you described above. Your script must accomplish the following:

1. **ID Mapping:** Parse the gtf file you downloaded earlier to create a mapping dictionary/table that links Transcript IDs (e.g., ENST...) to their parent Gene IDs (e.g., ENSG...).
2. **Aggregation:** Load Kallisto's abundance.tsv file, map the Transcript IDs to Gene IDs, and sum the est_counts for each gene.
3. **Merging:** Load the featureCounts output file, extract the Gene IDs and counts, and merge this dataframe with your aggregated Kallisto data.
4. **Statistics:** Calculate the Spearman rank correlation coefficient between the two pipelines.
5. **Visualisation:** Generate a scatter plot comparing the log_2(counts + 1) values from both tools and save it as a PDF or PNG.